

Kreacijski paterni

1. Singleton Patern

Singleton patern osigurava da klasa ima samo jednu instancu i pruža globalni pristup toj instanci. Ovaj patern je koristan kada je potrebno kontrolisati pristup dijeljenim resursima ili kada treba postojati samo jedan objekat određene klase u cijelom sistemu.

Singleton patern je u našem sistemu implementiran putem klase **SystemLogManager**, koja je zadužena za evidentiranje važnih događaja unutar sistema, kao što su prijava korisnika, kreiranje rezervacija, potvrda narudžbi i evidentiranje grešaka. Budući da više dijelova sistema može istovremeno koristiti funkcionalnost zapisivanja logova, korištenjem Singleton paterna obezbjeđuje se postojanje samo jedne instance klase SystemLogManager. Ovo omogućava dosljedno evidentiranje svih aktivnosti u sistemu.

2. Prototype Patern

Prototype patern služi za kreiranje objekata kloniranjem postojećih instanci umjesto kreiranja novih objekata od početka. Ovaj patern je koristan kada je kreiranje objekta skupo ili kompleksno, a potrebno je kreirati mnoge slične objekte.

Prototype patern je implementiran kroz sistem obavijesti kako bi se omogućilo efikasno kreiranje i slanje poruka kroz različite kanale komunikacije. Sistem često koristi slične šablone za slanje, kao što su predefinisani formati za Email ili SMS. Umjesto kreiranja novih objekata od nule za svaku pojedinačnu poruku, klasa Obavijest koristi metodu Clone() na postojećim prototipima (EmailPrototype ili SMSPrototype). Na taj način se kopira osnovna struktura šablona te se u nju dinamički ubacuje tekst konkretne poruke, čime se smanjuje ponavljanje koda i ubrzava procesiranje obavijesti u realnom vremenu.

3. Factory Method Patern

Factory Method patern služi za kreiranje objekata bez specifikovanja tačno koje klase će biti instancirane. Ovaj patern definiše interfejs za kreiranje objekata, ali dozvoljava podklasama da odluče koja klasa će biti instancirana. Na taj način se omogućava kreiranje objekata kroz zajednički interfejs, čime se povećava fleksibilnost sistema.

Factory Method patern može se implementirati u dijelu sistema koji se odnosi na kreiranje usluga. Budući da sistem podržava različite tipove usluga, kao što su šišanje i farbanje, koristi se apstraktna klasa **UslugaFactory** koja definiše metodu za kreiranje objekata usluge. Konkretne fabrike, poput SisanjeFactory i FarbanjeFactory, kreiraju odgovarajuće tipove usluga. Na ovaj način omogućava se jednostavno proširenje sistema dodavanjem novih usluga.

4. Abstract Factory Patern

Abstract Factory patern pruža interfejs za kreiranje familija povezanih ili zavisnih objekata bez specifikovanja njihovih konkretnih klasa. Ovaj patern je koristan kada sistem treba biti nezavisan od načina kreiranja, kompozicije i reprezentacije objekata koji ga čine.

Abstract Factory patern može se primijeniti u okviru sistema obavijesti za kreiranje porodice povezanih objekata. Na primjer, jedna fabrika može kreirati kompletan skup elemenata za email obavijesti, dok druga fabrika kreira odgovarajuće elemente za SMS obavijesti. Ovakav pristup omogućava jednostavno upravljanje različitim grupama povezanih objekata.

5. Builder Patern

Builder patern služi za kreiranje složenih objekata korak po korak. Omogućava kreiranje različitih reprezentacija objekta koristeći isti proces konstrukcije. Ovaj patern je posebno koristan kada objekat ima mnogo opcionalnih parametara ili kada kreiranje objekta zahtijeva složenu logiku.

Builder patern može se implementirati prilikom kreiranja objekta *Narudzba*. Budući da narudžba sadrži više elemenata njeno kreiranje može biti složeno.

Klasa **NarudzbaBuilder** omogućava postepeno dodavanje podataka i proizvoda prije konačnog formiranja objekta narudžbe te se na ovaj način pojednostavljuje proces kreiranja složenih objekata i povećava preglednost koda.